# Wholesale Supermarket Management System

Master Project & Viva Guide

Course: Database Management Systems (CS321 / DBMS Lab)

Department of Computer Science & Engineering
Indian Institute of Technology (BHU), Varanasi

**Group 23**

| | |
|---|---|
| **Domain:** | FMCG & commodity wholesale (Muzaffarnagar, UP) |
| **Database:** | MySQL 8.0 (InnoDB) |
| **Backend:** | Python 3.14, FastAPI, PyMySQL, JWT, Passlib (bcrypt) |
| **Frontend:** | Semantic HTML5, CSS, ES6 vanilla JavaScript |
| **Academic Year:** | 2026 |

Written so that every team member can understand and defend the whole project.

# Contents

**1 Executive Summary & Business Context** **3**
1.1 The business problem . . . 3
1.2 Core wholesale challenges and how we answer them . . . 3
1.3 System at a glance . . . 3

**2 High-Level System Architecture** **4**
2.1 Why this stack (viva justification) . . . 4

**3 Repository & Codebase Anatomy** **4**

**4 Relational Database Schema (14 Tables)** **4**
4.1 Summary of keys and constraints . . . 4
4.2 Table definitions . . . 5

**5 Relational Views (13 Views)** **7**
5.1 Financial views . . . 7
5.2 Inventory views . . . 7
5.3 Business intelligence views . . . 7

**6 Formal Normalization Analysis (1NF to BCNF)** **8**
6.1 First Normal Form (1NF) . . . 8
6.2 Second Normal Form (2NF) . . . 8
6.3 Third Normal Form (3NF) . . . 8
6.4 Boyce-Codd Normal Form (BCNF) . . . 8

**7 Transactions, Concurrency & ACID** **8**
7.1 The "last unit" race condition . . . 8
7.2 Solution: pessimistic row locking . . . 9
7.3 Verification: `tests/test_concurrency.py` . . . 9
7.4 ACID in this system . . . 9

**8 Stored Procedures & FEFO Allocation** **9**
8.1 The FEFO algorithm . . . 9
8.2 `sp_create_bill` . . . 9

**9 Core Business Logic & Financial Accounting** **10**
9.1 Pricing and tax formulae . . . 10
9.2 Loyalty reward engine . . . 11
9.3 Demand-driven stock requests . . . 11

**10 Security Architecture & Threat Defense** **11**
10.1 Password cryptography . . . 11
10.2 Stateless authentication (JWT) . . . 11

10.3 Role-based access control . . . . . 11
10.4 SQL injection defense . . . . . 11

**11 Three-Role Operational Workflows** **12**
11.1 Customer portal . . . . . 12
11.2 Cashier POS terminal (`staff/pos.html`) . . . . . 12
11.3 Owner dashboard and BI . . . . . 12

**12 Master Viva Grill: 25 Questions & Model Answers** **12**

**13 Quick Reference & Demo Cheatsheet** **15**
13.1 Demo credentials . . . . . 15
13.2 Shell commands . . . . . 15

**14 Design Notes & Known Limitations** **15**

# 1 Executive Summary & Business Context

## 1.1 The business problem

Standard retail Point of Sale (POS) software fails for wholesale agricultural and commodity trading. A wholesale dealer in Muzaffarnagar (Western UP) deals with four product categories:

- **Kriyana** (groceries and grains): Basmati rice, refined sugar, mustard oil, spices.
- **Dryfruits:** almonds, cashews, pistachios, raisins.
- **Desi Jadi Booti** (herbal / Ayurvedic commodities): ashwagandha roots, dried tulsi leaves, mulethi.
- **Tota Rang Gulal:** Holi colours and herbal gulal pigments.

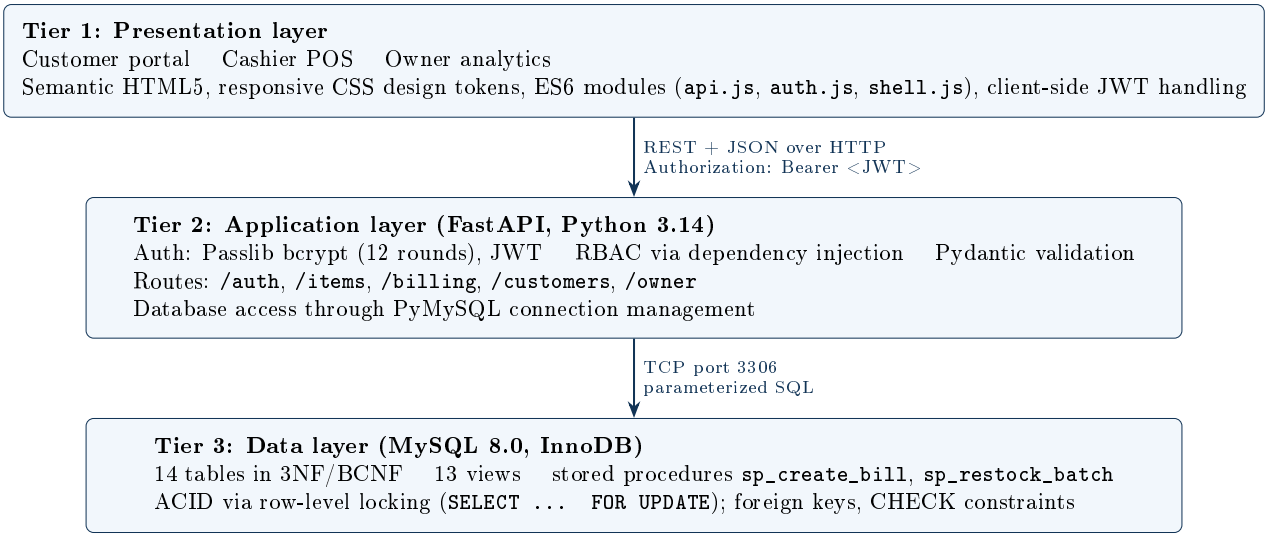
## 1.2 Core wholesale challenges and how we answer them

| Challenge | Why it matters | Our design answer |
|---|---|---|
| Price volatility | Prices change batch by batch with mandi arrivals; one item can have several batches bought at different rates. | Price lives in `Batch`, not `Item`. |
| Perishability | Spices, dryfruits and herbs expire; dead stock is a direct loss. | FEFO allocation (First-Expired, First-Out) using `Expiry_Date`. |
| GST compliance | Rates of 0%, 5%, 12%, 18%; intra-state sales split 50/50 into CGST and SGST with HSN codes on the invoice. | Insert-only `HSN_Tax_Rate` table; split computed in views. |
| Audit-proof financials | Stored totals drift when quantities or rates change (update anomalies). | Bill totals and line taxes are **never stored**; they are derived by views. |
| Concurrency | Many cashiers bill at once; the last 10 kg of cashews must not be sold twice. | Transactions with `SELECT ... FOR UPDATE` row locks. |

## 1.3 System at a glance

- 3-tier architecture: browser UI → FastAPI REST API → MySQL 8.0.
- 14 normalized tables (3NF/BCNF) and 13 views.
- Stored procedure `sp_create_bill`: atomic FEFO billing with loyalty handling.
- Three roles: **Owner**, **Employee** (cashier), **Customer**, with JWT-based RBAC.

# 2 High-Level System Architecture




### 2.1 Why this stack (viva justification)

- **MySQL 8.0 InnoDB:** real ACID transactions, row-level locking, views, CHECK constraints.
- **FastAPI:** asynchronous, automatic OpenAPI/Swagger docs at `/docs`, strong typing via Pydantic, dependency-injected security.
- **Vanilla ES6 modules:** no Node.js, webpack or npm; the frontend runs directly in the browser and is easy to debug.

## 3 Repository & Codebase Anatomy

```
DBMSProject/
+-- db/
|   +-- 01_schema.sql       # 14 tables, PKs, FKs, CHECKs, indexes
|   +-- 02_views.sql        # 13 views (financial, inventory, analytics)
|   +-- 03_seed.sql         # realistic Muzaffarnagar wholesale seed data
|   +-- 04_procedures.sql   # sp_create_bill, sp_restock_batch
+-- app/
|   +-- main.py             # FastAPI entrypoint, CORS, route registration
|   +-- database.py         # MySQL connection pool and cursor management
|   +-- services/auth_service.py   # bcrypt hashing, JWT, RBAC guards
|   +-- routes/
|       +-- auth_routes.py       # login, profile
|       +-- item_routes.py       # catalog, categories, batches
|       +-- billing_routes.py    # POS checkout, invoices
|       +-- customer_routes.py   # portal, requests, feedback, loyalty
|       +-- owner_routes.py      # BI reports, inventory, employees
+-- frontend/               # served by FastAPI under /app
|   +-- index.html          # landing / login
|   +-- customer/  staff/  owner/   # role-specific pages
|   +-- js/                 # api.js, auth.js, shell.js, ui.js
|   +-- css/                # tokens.css, base.css, layout.css, components.css
+-- tests/
|   +-- test_concurrency.py # multi-threaded proof of row locking
|   +-- test_loyalty.py     # loyalty calculation and limit tests
|   +-- test_schema.py      # schema invariants and views
+-- DEMO.md  ER_DIAGRAM.md  PROJECT_MASTER_GUIDE.md/.pdf  requirements.txt
```

## 4 Relational Database Schema (14 Tables)

The database `supermarket_db` has 14 tables normalized to 3NF/BCNF.

### 4.1 Summary of keys and constraints

| Table | Primary key | Foreign keys / key constraints |
|---|---|---|
| 1. Category | `Category_ID` | UNIQUE(`Category_Name`) |
| 2. Supplier | `Supplier_ID` | UNIQUE(`Phone_Number`), UNIQUE(`GSTIN`) |
| 3. HSN_Tax_Rate | `HSN_Code` | Insert-only; `Tax_Rate` in percent (0, 5, 12, 18) |
| 4. Item | `Item_ID` | FK → Category; UNIQUE(`Item_Name`) |
| 5. Batch | `Batch_ID` | FK → Item, Supplier, HSN_Tax_Rate; CHECK(`Live_Stock >= 0`) |
| 6. Customer | `Customer_ID` | UNIQUE(`Phone_Number`); CHECK(`Loyalty_Points >= 0`) |
| 7. Employee | `Employee_ID` | UNIQUE(`Phone_Number`), UNIQUE(`Aadhaar_Number`) |
| 8. Users | `User_ID` | UNIQUE(`Username`); FK → Employee, Customer; Role ENUM(Owner, Employee, Customer) |
| 9. Bill | `Bill_ID` | FK → Customer (NULL for walk-ins), FK → Employee (`Billed_By_Employee_ID`) |
| 10. Bill_Details | (`Bill_ID`, `Batch_ID`) | Composite PK; FK → Bill, Batch; CHECK(`Quantity > 0`) |
| 11. Stock_Request | `Request_ID` | FK → Customer, Item, Employee; XOR CHECK on item reference |
| 12. Loyalty_Transaction | `Transaction_ID` | FK → Customer, Bill; `Points_Change` is signed |
| 13. Feedback | `Feedback_ID` | FK → Customer, Bill; CHECK(`Rating BETWEEN 1 AND 5`) |
| 14. Settings | `Setting_Key` | Key-value system configuration |

## 4.2 Table definitions

### 1. Category

Top-level commodity groups. `Category_ID` (INT, AUTO_INCREMENT, PK); `Category_Name` (VARCHAR(50), NOT NULL, UNIQUE); `Description` (TEXT).

### 2. Supplier

Wholesale vendors. `Supplier_ID` (PK); `Supplier_Name` (VARCHAR(100), NOT NULL); `Contact_Person`; `Phone_Number` (VARCHAR(15), NOT NULL, UNIQUE); `Email`; `Address` (TEXT); `GSTIN` (VARCHAR(15), UNIQUE; the 15-character GST identification number).

### 3. HSN_Tax_Rate (insert-only)

`HSN_Code` (VARCHAR(10), PK); `Tax_Rate` (DECIMAL(5,2), NOT NULL, CHECK $\geq 0$); `Description`; `Effective_From` (DATE, NOT NULL).

**Architectural rule**

This table is strictly **insert-only**. When the government changes a rate, a new row is inserted; existing rows are never updated or deleted. This preserves historical billing integrity.

### 4. Item

The abstract product (for example "Basmati Rice 1121"). `Item_ID` (PK); `Item_Name` (NOT NULL, UNIQUE); `Category_ID` (FK, NOT NULL); `Unit` (e.g. kg, litre, packet); `Reorder_Level` (INT, default 10). **Item holds no price and no tax rate;** those belong to `Batch`.

### 5. Batch

A physical stock lot arriving at the godown. `Batch_ID` (PK); `Item_ID`, `Supplier_ID`, `HSN_Code` (all FK, NOT NULL); `Batch_Number`; `Purchase_Price` (cost from supplier); `Wholesale_Price` (selling price, tax-exclusive); `Initial_Stock` (CHECK $\geq 0$); `Live_Stock` (CHECK $\geq 0$, decremented on sale); `Received_Date`; `Expiry_Date` (drives FEFO); `Is_Active` (BOOLEAN, default TRUE).

### 6. Customer

`Customer_ID` (PK); `Customer_Name`; `Phone_Number` (UNIQUE); `Email`; `Address`; `GSTIN` (optional, for B2B buyers); `Loyalty_Points` (INT, default 0, CHECK $\geq 0$; running balance).

### 7. Employee

`Employee_ID` (PK); `First_Name`; `Last_Name`; `Phone_Number` (UNIQUE); `Email`; `Role` (Cashier, Manager, Store Staff); `Salary`; `Aadhaar_Number` (VARCHAR(12), UNIQUE); `Is_Active`.

### 8. Users

Authentication and RBAC. `User_ID` (PK); `Username` (UNIQUE); `Password_Hash` (VARCHAR(255), bcrypt, 12 rounds); `Role` ENUM(Owner, Employee, Customer); `Employee_ID` and `Customer_ID` (nullable FKs); `Last_Login`; `Is_Active`.

### 9. Bill

Invoice header. `Bill_ID` (PK); `Bill_Number` (UNIQUE); `Customer_ID` (nullable FK; NULL means walk-in); `Billed_By_Employee_ID` (FK, NOT NULL); `Bill_Date` (DATETIME, default now); `Payment_Method` ENUM(Cash, UPI, Card, Credit); `Loyalty_Points_Redeemed` (CHECK $\geq 0$). **Subtotal, GST and net amount are not stored here;** they are computed in `v_bill_totals`.

### 10. Bill_Details

Invoice line items: `Bill_ID`, `Batch_ID`, `Quantity` (CHECK $> 0$), PRIMARY KEY (`Bill_ID`, `Batch_ID`). The unit price is *not* duplicated; it is joined from `Batch.Wholesale_Price`.

### 11. Stock_Request

Demand-driven procurement requests. `Request_ID` (PK); `Customer_ID` (FK); `Item_ID` (nullable FK, for re-stocking an existing item); `Requested_Item_Name` (nullable, for a brand-new product); `Quantity_Requested` (CHECK $> 0$); `Request_Date`; `Status` ENUM(Pending, Approved, Rejected, Fulfilled); `Remarks`; `Handled_By_Employee_ID` (nullable FK).

```
CHECK ((Item_ID IS NOT NULL) <> (Requested_Item_Name IS NOT NULL))
```

A request names *either* an existing item *or* a new item, never both and never neither (exclusive OR).

### 12. Loyalty_Transaction

Immutable audit ledger of reward points: `Transaction_ID` (PK); `Customer_ID`; `Bill_ID` (nullable); `Points_Change` (positive when earned, negative when redeemed); `Transaction_Date`; `Reason`.

### 13. Feedback

`Feedback_ID` (PK); `Customer_ID`; `Bill_ID`; `Rating` (TINYINT, 1 to 5); `Comments`; `Created_At`.

#### 14. Settings

`Setting_Key` (PK); `Setting_Value`; `Description`. Key parameters: `Loyalty_Earn_Rate` (100.00), `Loyalty_Redeem_Rate` (1.00), `Loyalty_Max_Discount_Percent` (50.00).

# 5 Relational Views (13 Views)

Views give abstraction, enforce business rules and avoid redundant stored data.

## 5.1 Financial views

**1.** `v_bill_lines` computes price, GST rate, CGST, SGST and total for each bill line.

```
CREATE VIEW v_bill_lines AS
SELECT bd.Bill_ID, bd.Batch_ID, i.Item_Name, b.Batch_Number, bd.Quantity,
  b.Wholesale_Price AS Unit_Price,
  (bd.Quantity * b.Wholesale_Price) AS Line_Subtotal,
  htr.Tax_Rate AS GST_Rate,
  ROUND((bd.Quantity * b.Wholesale_Price) * (htr.Tax_Rate / 100.0) / 2.0, 2) AS CGST_Amount,
  ROUND((bd.Quantity * b.Wholesale_Price) * (htr.Tax_Rate / 100.0) / 2.0, 2) AS SGST_Amount,
  ROUND((bd.Quantity * b.Wholesale_Price) * (1.0 + htr.Tax_Rate / 100.0), 2) AS Line_Total
FROM Bill_Details bd
JOIN Batch b ON bd.Batch_ID = b.Batch_ID
JOIN Item i ON b.Item_ID = i.Item_ID
JOIN HSN_Tax_Rate htr ON b.HSN_Code = htr.HSN_Code;
```

**2.** `v_bill_totals` aggregates the lines into the fiscal summary of each bill.

```
CREATE VIEW v_bill_totals AS
SELECT b.Bill_ID, b.Bill_Number, b.Bill_Date, b.Customer_ID, c.Customer_Name,
  b.Billed_By_Employee_ID,
  CONCAT(e.First_Name, ' ', e.Last_Name) AS Cashier_Name,
  b.Payment_Method, b.Loyalty_Points_Redeemed,
  CAST(b.Loyalty_Points_Redeemed AS DECIMAL(10,2)) AS Loyalty_Discount,
  COALESCE(SUM(vbl.Line_Subtotal), 0.00) AS Subtotal,
  COALESCE(SUM(vbl.CGST_Amount), 0.00) AS CGST_Total,
  COALESCE(SUM(vbl.SGST_Amount), 0.00) AS SGST_Total,
  COALESCE(SUM(vbl.CGST_Amount + vbl.SGST_Amount), 0.00) AS GST_Total,
  COALESCE(SUM(vbl.Line_Total), 0.00) AS Gross_Amount,
  GREATEST(0.00, COALESCE(SUM(vbl.Line_Total), 0.00)
    - CAST(b.Loyalty_Points_Redeemed AS DECIMAL(10,2))) AS Net_Payable
FROM Bill b
LEFT JOIN Customer c ON b.Customer_ID = c.Customer_ID
JOIN Employee e ON b.Billed_By_Employee_ID = e.Employee_ID
LEFT JOIN v_bill_lines vbl ON b.Bill_ID = vbl.Bill_ID
GROUP BY b.Bill_ID;
```

## 5.2 Inventory views

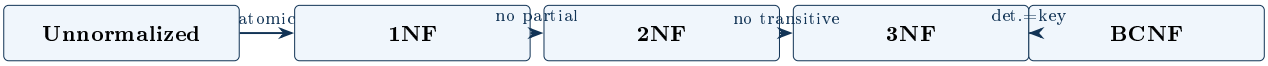
- **3.** `v_active_batches`: batches available for sale (`Live_Stock > 0` and `Expiry_Date >= CURDATE()`).
- **4.** `v_expiring_batches`: batches expiring within 30 days, for promotions.
- **5.** `v_low_stock`: total active stock per item compared with `Reorder_Level`.
- **6.** `v_stock_summary`: units on hand and capital tied up per item.

## 5.3 Business intelligence views

- **7.** `v_daily_sales`: daily revenue, bill counts and tax collected.
- **8.** `v_monthly_sales`: monthly trends for reporting.
- **9.** `v_category_sales`: revenue per commodity category.
- **10.** `v_top_selling_items`: items ranked by quantity and revenue.
- **11.** `v_customer_summary`: lifetime value, order count, loyalty balance.

- **12.** `v_supplier_purchases`**:** acquisition cost per supplier.
- **13.** `v_gst_summary`**:** CGST and SGST collected per HSN class (tax reconciliation).

# 6 Formal Normalization Analysis (1NF to BCNF)




## 6.1 First Normal Form (1NF)

Every attribute holds atomic values; there are no repeating groups or comma-separated lists. Line items are decomposed into the junction table `Bill_Details` instead of being packed into a column of `Bill`.

## 6.2 Second Normal Form (2NF)

Must be in 1NF, and every non-prime attribute must depend on the *whole* candidate key. In `Bill_Details` the key is (Bill_ID, Batch_ID) and the only non-key attribute is `Quantity`:

$$(\text{Bill_ID}, \text{Batch_ID}) \rightarrow \text{Quantity}$$

Neither attribute alone determines `Quantity`, so there is no partial dependency. `Item_Name`, `Wholesale_Price` and `Tax_Rate` are deliberately *not* in `Bill_Details`; they would depend on `Batch_ID` alone (a partial dependency) and violate 2NF.

## 6.3 Third Normal Form (3NF)

Must be in 2NF, and no non-prime attribute may be transitively dependent on a key.

- A flat design would have Batch_ID $\rightarrow$ HSN_Code $\rightarrow$ Tax_Rate, a transitive dependency. We resolve it by moving the rate into its own relation `HSN_Tax_Rate(HSN_Code, Tax_Rate, ...)`; `Batch` keeps only the `HSN_Code` foreign key.
- `Item`: Item_ID $\rightarrow$ (Item_Name, Category_ID, Unit). `Batch`: Batch_ID $\rightarrow$ (Item_ID, Supplier_ID, HSN_Code, Wholesale_Price, ...).
- **Derivable attributes:** storing `Subtotal` or `Net_Amount` in `Bill` makes it depend on aggregated `Bill_Details` rows. Changing a quantity or a rate would leave the stored total inconsistent (an update anomaly). Computing totals in `v_bill_totals` avoids this.

## 6.4 Boyce-Codd Normal Form (BCNF)

For every non-trivial dependency $X \rightarrow Y$, $X$ must be a superkey. In all 14 tables, each determinant is either the primary key or a candidate key enforced by a UNIQUE index (e.g. `Supplier.Phone_Number`, `Users.Username`, `Customer.Phone_Number`). Hence the schema satisfies BCNF.

# 7 Transactions, Concurrency & ACID

## 7.1 The "last unit" race condition

Suppose 10 units of cashews remain and two cashiers bill 10 units simultaneously *without* locking:

1. Cashier 1 reads `Live_Stock = 10` (sufficient). Cashier 2 also reads 10 (sufficient).
2. Both set `Live_Stock = 10 - 10 = 0`.
3. Both print invoices: 20 units sold from 10 physical units.

The result is overselling, an inventory deficit and a classic **lost update** anomaly.

### 7.2 Solution: pessimistic row locking

```
SELECT Live_Stock, Expiry_Date
FROM Batch
WHERE Batch_ID = ?
FOR UPDATE;
```

`FOR UPDATE` takes an exclusive (X) lock on the batch row inside the transaction. Cashier 2's transaction waits until Cashier 1 issues `COMMIT` or `ROLLBACK`. When it resumes it reads `Live_Stock = 0`, detects insufficient stock and rejects the second bill.

### 7.3 Verification: `tests/test_concurrency.py`

The test creates a batch with exactly 1 unit and starts 2 worker threads that check out concurrently. **Observed result:** exactly one thread succeeds (HTTP 200) and the other fails with HTTP 400 (*Insufficient stock*). No double allocation occurs.

### 7.4 ACID in this system

| Property | Mechanism |
|---|---|
| Atomicity | Inserting `Bill` and `Bill_Details`, deducting `Live_Stock`, adjusting loyalty points and writing `Loyalty_Transaction` all happen in *one* transaction. Any failure triggers `ROLLBACK`, leaving no orphan rows. |
| Consistency | The engine enforces CHECK (`Live_Stock >= 0`), foreign keys (`ON DELETE RESTRICT`) and the Stock_Request XOR rule; violating transactions abort. |
| Isolation | InnoDB's default REPEATABLE READ (MVCC) plus explicit `SELECT ... FOR UPDATE` prevents dirty reads, non-repeatable reads and lost updates on inventory rows. |
| Durability | InnoDB write-ahead logging (redo log). After `COMMIT` the changes survive crashes and power loss. |

## 8 Stored Procedures & FEFO Allocation

### 8.1 The FEFO algorithm

When a cashier bills 25 kg of Basmati rice, several batches may exist:

- Batch 101: 10 kg left, expires Dec 2026.
- Batch 102: 20 kg left, expires Mar 2027.

FEFO allocation: order active batches by `Expiry_Date ASC`; take 10 kg from Batch 101 (now 0) and the remaining 15 kg from Batch 102 (5 kg left); insert two `Bill_Details` rows, one per batch. Nothing near expiry is left unsold.

### 8.2 `sp_create_bill`

Located in `db/04_procedures.sql`; it wraps the whole checkout in one MySQL transaction. The FEFO loop (step 4) is summarized as a comment here.

```
DELIMITER //
CREATE PROCEDURE sp_create_bill(
  IN p_customer_id INT, IN p_employee_id INT, IN p_payment_method VARCHAR(20),
  IN p_redeem_points INT,
  IN p_items_json JSON,   -- [{"item_id": 1, "qty": 10}, ...]
  OUT p_bill_id INT, OUT p_status_message VARCHAR(255))
proc_body: BEGIN
  DECLARE v_bill_number VARCHAR(50);
  DECLARE v_net_total DECIMAL(10,2) DEFAULT 0.00;
```

```
  DECLARE v_earned_points INT DEFAULT 0;
  DECLARE v_cust_points INT DEFAULT 0;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SET p_bill_id = NULL;
    SET p_status_message = 'Transaction rolled back due to error.';
  END;

  START TRANSACTION;
  -- 1. Validate loyalty points if redeeming (lock the customer row)
  IF p_customer_id IS NOT NULL AND p_redeem_points > 0 THEN
    SELECT Loyalty_Points INTO v_cust_points FROM Customer
    WHERE Customer_ID = p_customer_id FOR UPDATE;
    IF p_redeem_points > v_cust_points THEN
      ROLLBACK;
      SET p_bill_id = NULL;
      SET p_status_message = 'Cannot redeem more points than customer balance.';
      LEAVE proc_body;
    END IF;
  END IF;

  -- 2. Generate bill number
  SET v_bill_number = CONCAT('BILL-', DATE_FORMAT(NOW(), '%Y%m%d%H%i%s'));

  -- 3. Insert bill header
  INSERT INTO Bill (Bill_Number, Customer_ID, Billed_By_Employee_ID,
                    Payment_Method, Loyalty_Points_Redeemed)
  VALUES (v_bill_number, p_customer_id, p_employee_id, p_payment_method,
          COALESCE(p_redeem_points, 0));
  SET p_bill_id = LAST_INSERT_ID();

  -- 4. FEFO: iterate items, lock batches (FOR UPDATE), deduct stock,
  --    insert Bill_Details rows.   [loop omitted for brevity]

  -- 5. Earned points: 1 point per Rs 100 of net paid
  SELECT Net_Payable INTO v_net_total FROM v_bill_totals WHERE Bill_ID = p_bill_id;
  SET v_earned_points = FLOOR(v_net_total / 100);

  -- 6. Update loyalty ledger and balance
  IF p_customer_id IS NOT NULL THEN
    IF p_redeem_points > 0 THEN
      INSERT INTO Loyalty_Transaction (Customer_ID, Bill_ID, Points_Change, Reason)
      VALUES (p_customer_id, p_bill_id, -p_redeem_points, 'Redeemed at checkout');
    END IF;
    IF v_earned_points > 0 THEN
      INSERT INTO Loyalty_Transaction (Customer_ID, Bill_ID, Points_Change, Reason)
      VALUES (p_customer_id, p_bill_id, v_earned_points, 'Earned from purchase');
    END IF;
    UPDATE Customer
    SET Loyalty_Points = Loyalty_Points - COALESCE(p_redeem_points, 0) + v_earned_points
    WHERE Customer_ID = p_customer_id;
  END IF;

  COMMIT;
  SET p_status_message = 'Bill generated successfully.';
END //
DELIMITER ;
```

# 9 Core Business Logic & Financial Accounting

## 9.1 Pricing and tax formulae

Wholesale sales are priced tax-exclusive:

$$\begin{aligned}
\text{Line Subtotal} &= \text{Quantity} \times \text{Wholesale Price} \\
\text{Line GST} &= \text{Line Subtotal} \times \frac{\text{GST Rate}}{100} \\
\text{CGST} = \text{SGST} &= \frac{\text{Line GST}}{2} \\
\text{Line Total} &= \text{Line Subtotal} + \text{Line GST} \\
\text{Gross Amount} &= \sum \text{Line Total} \\
\text{Net Payable} &= \max\bigl(0,\ \text{Gross Amount} - \text{Loyalty Discount}\bigr)
\end{aligned}$$

### 9.2 Loyalty reward engine

- **Earn rate:** 1 point per Rs 100 of net amount paid.
- **Redemption:** 1 point = Rs 1 discount on the invoice.
- **Cap:** at most 50% of the gross bill (protects dealer margin).
- **Walk-ins** (`Customer_ID = NULL`) neither earn nor redeem.

**Three-layer guard against over-redemption:**

1. **UI:** the POS input is clamped to min(balance, 50% of gross).
2. **API:** the FastAPI route compares points with the live balance and returns HTTP 400 if exceeded.
3. **Database:** `sp_create_bill` locks the customer row, and rolls back with an error message if points exceed the balance.

### 9.3 Demand-driven stock requests

Customers can request out-of-stock or unlisted products. The XOR constraint means `Item_ID` is set *or* `Requested_Item_Name` is set. **Smart auto-matching:** if a customer types "basmati rice", the backend does a case-insensitive lookup, resolves it to the existing `Item_ID` and stores it as a catalog request.

## 10 Security Architecture & Threat Defense

### 10.1 Password cryptography

Passwords are never stored in plain text. We use bcrypt (via Passlib) with 12 cost rounds and a unique random 128-bit salt per password, which defeats rainbow tables; the cost factor deliberately slows brute force.

### 10.2 Stateless authentication (JWT)

`POST /auth/login` validates credentials and returns a signed JWT (HS256) that expires after 8 hours:

```
{ "sub": "owner", "role": "Owner", "user_id": 1, "exp": 1793498400 }
```

Clients send it as `Authorization: Bearer <token>`.

### 10.3 Role-based access control

```
def require_role(allowed_roles: list[str]):
    def dependency(token_data: dict = Depends(get_current_user)):
        if token_data["role"] not in allowed_roles:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
                                detail="Access denied: Insufficient permissions.")
        return token_data
    return dependency
```

- `/owner/*` requires `['Owner']`.
- `/billing/*` requires `['Owner', 'Employee']`.
- `/customers/me/*` requires `['Customer']`.

### 10.4 SQL injection defense

No raw string concatenation; every query is parameterized through PyMySQL:

```
cursor.execute(
    "SELECT * FROM Item WHERE Item_Name LIKE %s AND Category_ID = %s",
    (f"%{query}%", category_id))
```

User input is sent separately from the SQL text and is treated purely as data.

# 11 Three-Role Operational Workflows

## 11.1 Customer portal

- **Dashboard** (`customer/home.html`): loyalty balance, profile, recent activity.
- **Catalog** (`catalog.html`): category filters, wholesale rates, stock badges (In stock, Low stock, Out of stock).
- **Stock requests** (`requests.html`): request restocks or suggest new commodities.
- **Invoices** (`bills.html`, `invoice.html`): past purchases and printable GST invoices with CGST/SGST breakdown.
- **Feedback** (`feedback.html`): 1 to 5 star rating with comments.

## 11.2 Cashier POS terminal (`staff/pos.html`)

- Customer phone lookup shows name and loyalty balance.
- Cart with live unit price, live stock validation and line subtotals.
- Payment methods: Cash, UPI, Card, Credit.
- Loyalty redemption with clamping and real-time balance check.
- One-click atomic checkout: creates the bill, deducts FEFO stock, updates loyalty, opens the printable invoice.

## 11.3 Owner dashboard and BI

- **Dashboard:** KPIs (total bills, low stock, pending requests, expiring batches), 30-day revenue chart, top 5 items.
- **Inventory & batches:** add incoming batches, adjust stock, track supplier lots.
- **Stock requests:** approve or reject customer requests.
- **Seven reports:** (1) sales trends, (2) category performance, (3) top-selling items, (4) low-stock alert, (5) expiring batches, (6) customer valuation, (7) GST tax summary.

# 12 Master Viva Grill: 25 Questions & Model Answers

**Q1. Why MySQL 8.0 rather than a NoSQL database like MongoDB?**

Wholesale trade needs financial accounting, stock deduction and statutory GST compliance, which demand ACID guarantees, referential integrity and joins. In a denormalized document store, updating an HSN rate or deducting stock atomically across concurrent cashiers needs complex application-level locking and risks lost updates. InnoDB gives row locking (`SELECT ... FOR UPDATE`), rollback, foreign keys and views natively.

**Q2. Why are Subtotal, GST and Total not stored in `Bill`?**

They are derivable attributes. Storing them creates a dependency on the aggregated lines, so a corrected quantity or changed rate would make the stored total diverge (update anomaly, a 3NF concern). `v_bill_totals` computes them on the fly from `Bill_Details`, `Batch` and `HSN_Tax_Rate`, so they are always consistent.

**Q3. Why is `HSN_Tax_Rate` insert-only?**

GST rates change by notification. If a rate were updated in place (say HSN 1006 from 5% to 12%), every historical bill would retroactively show 12% through the views and falsify past filings. With insert-only

rows, each batch keeps its foreign key to the rate valid when it was purchased, giving an unalterable audit trail.

**Q4. Explain the race condition on the last unit and your fix.**

Two cashiers both read `Live_Stock = 1`, both deduct, and two invoices are printed for one physical unit (lost update / double allocation). We run the deduction in a transaction with `SELECT Live_Stock FROM Batch WHERE Batch_ID = ? FOR UPDATE`, which takes an exclusive row lock. The second transaction waits, then reads 0, raises an error and the bill is rejected.

**Q5. Difference between 3NF and BCNF? Is your schema in BCNF?**

3NF: for every dependency $X \to Y$, either $X$ is a superkey or $Y$ is a prime attribute. BCNF removes the second exception: $X$ must always be a superkey. In our schema every determinant is a primary key or an enforced UNIQUE candidate key (e.g. `Phone_Number`, `Username`), so it is in BCNF.

**Q6. What is FEFO and why not FIFO?**

FIFO sells by arrival date; FEFO sells by `Expiry_Date ASC`. A newer batch can expire earlier because of harvest variation, so FEFO clears the stock nearest to expiry first and minimizes spoilage.

**Q7. How is the XOR constraint in `Stock_Request` implemented?**

A request must reference an existing item or propose a new name, never both or neither. In MySQL 8: `CHECK ((Item_ID IS NOT NULL) <> (Requested_Item_Name IS NOT NULL))`. Each IS NOT NULL test yields 1 or 0, so `<>` is true exactly when the two differ, which is XOR.

**Q8. How do you stop a customer redeeming more points than they have?**

Three layers: the POS UI clamps the input to min(points, 50% of gross); the API (`billing_routes.py`) reads the live balance and returns HTTP 400 if exceeded; and `sp_create_bill` locks the customer row with `FOR UPDATE`, compares, and rolls back with an error message if the request is too large.

**Q9. Why split `Item` and `Batch`?**

One item has many lots over time, each with its own supplier, dates, cost, price and stock. Merging them would repeat `Item_Name`, `Category_ID` and `Unit` in every lot (redundancy, violating 2NF/3NF). Splitting gives a clean 1:N relationship.

**Q10. How do you prevent SQL injection?**

All queries are parameterized (`%s` placeholders in PyMySQL); inputs are never concatenated into SQL. Even input such as `' OR '1'='1` is treated as a literal string value.

**Q11. How are passwords stored?**

With bcrypt through Passlib, 12 cost rounds and a random 128-bit salt per password. The adjustable work factor slows brute force and GPU attacks. Plain-text passwords are never stored or logged.

**Q12. Purpose of `v_bill_lines`?**

It breaks each invoice into lines with unit price, subtotal, GST rate, CGST, SGST and line total by joining `Bill_Details`, `Batch`, `Item` and `HSN_Tax_Rate`. It feeds both the printable invoice and `v_bill_totals`.

**Q13. If a customer or employee leaves, are past bills deleted?**

No. Foreign keys from `Bill` use `ON DELETE RESTRICT`, so referenced customers and employees cannot be physically deleted. The application also uses soft deletes (`Is_Active = FALSE`) to keep the audit trail.

**Q14. How are walk-in customers handled?**

`Bill.Customer_ID` is nullable. Walk-in sales store NULL, print a valid cash memo and skip loyalty earning and redemption.

**Q15. Why is the key of `Bill_Details` composite?**

`Bill_Details` resolves the many-to-many relation between `Bill` and `Batch`. The composite key (`Bill_ID`, `Batch_ID`) allows a batch at most once per bill (quantities are consolidated) and enforces entity integrity.

**Q16. How do you avoid deadlocks?**

Deadlocks come from opposite lock orders (T1 locks A then B, T2 locks B then A). We make every transaction acquire row locks in one deterministic order. Because FEFO itself orders batches by expiry (with `Batch_ID` as tie-breaker), all concurrent bills lock the same batches in the same sequence, so circular waits cannot form.

**Q17. Role of foreign-key indexes in InnoDB?**

Joins such as `Bill` to `Bill_Details` use the index for $O(\log N)$ B+ tree lookups instead of $O(N)$ scans. InnoDB also requires an index on foreign key columns to check referential integrity efficiently on INSERT, UPDATE and DELETE.

**Q18. Which isolation level is used?**

InnoDB's default REPEATABLE READ, using MVCC for consistent reads and gap locks for locking reads, which prevents dirty and non-repeatable reads (and phantoms for locking reads). We add explicit `FOR UPDATE` locks on the inventory rows to get serialization where it matters, without locking whole tables.

**Q19. How is Indian GST represented?**

`HSN_Tax_Rate` stores the total rate (e.g. 5.00). For intra-state sales (within Uttar Pradesh) it is split equally: `CGST_Amount = ROUND(Subtotal * (Tax_Rate / 100.0) / 2.0, 2)` and the same for SGST, computed in `v_bill_lines`.

**Q20. Views versus stored tables?**

Views: no redundancy, no update anomalies, always consistent; cost is on-the-fly computation. Stored tables: faster reads on huge archives, but risk of drift under concurrent writes or corrections. For tens of thousands of bills with indexed joins, views answer in milliseconds, so views are the better choice for financials.

**Q21. DELETE vs TRUNCATE vs DROP?**

`DELETE` (DML) removes rows matching a WHERE clause, is logged per row, fires triggers and can be rolled back. `TRUNCATE` (DDL) empties the whole table quickly, resets AUTO_INCREMENT, causes an implicit commit in MySQL (no rollback) and fails if foreign keys reference the table. `DROP` (DDL) removes the table structure, indexes, constraints and data.

**Q22. Purpose of the `Settings` table?**

It keeps business rules (loyalty earn rate, redemption cap) as data instead of hard-coded constants, so the owner can adjust policy from the dashboard without redeploying code.

**Q23. How does the frontend talk to the backend securely?**

Through `fetch` wrapped in `frontend/js/api.js`. After login the JWT is kept in `sessionStorage` and attached as `Authorization: Bearer` on each call. On HTTP 401 the frontend clears storage and redirects to login. (In production this should run over HTTPS.)

**Q24. Clustered vs secondary indexes in InnoDB?**

The primary key is the clustered index: its B+ tree leaves hold the full rows, one per table. Secondary indexes (e.g. UNIQUE phone, foreign keys) store the indexed values plus the primary key, so a lookup does a second step into the clustered index to fetch the row.

**Q25. How would you scale to 50 regional branches?**

(1) Add a `Branch` table and `Branch_ID` foreign keys in `Batch`, `Employee`, `Bill`, `Stock_Request`. (2) Primary–replica replication: writes (POS) to the primary, BI reports to read replicas. (3) Shard historical bills by date or branch. (4) Cache catalog, categories and HSN rates in Redis.

# 13 Quick Reference & Demo Cheatsheet

## 13.1 Demo credentials

| Role | Username | Password | Use in demo |
|---|---|---|---|
| Owner | `owner` | `owner123` | Analytics, alerts, approvals, employees |
| Employee | `ramesh_emp` | `ramesh123` | POS billing, FEFO, customer lookup |
| Customer | `anil_cust` | `anil123` | Catalog, bills, stock requests |

## 13.2 Shell commands

```
# 1. Start the backend
python -m uvicorn app.main:app --reload --port 8000

# 2. Open in browser
#   App:        http://localhost:8000/app/index.html
#   Swagger:    http://localhost:8000/docs
#   Owner:      http://localhost:8000/app/owner/dashboard.html
#   POS:        http://localhost:8000/app/staff/pos.html
#   Customer:   http://localhost:8000/app/customer/home.html

# 3. Concurrency stress test
python tests/test_concurrency.py

# 4. Check the database
mysql -u root -p supermarket_db -e "SELECT count(*) FROM Bill;"
```

# 14 Design Notes & Known Limitations

Good to know before the viva; examiners often probe exactly these points.

1. **Settings vs. the procedure.** `sp_create_bill` computes earned points with a literal `/ 100`, so editing `Loyalty_Earn_Rate` in `Settings` does not change the procedure unless it is rewritten to read that row. Q22 describes the intended design.
2. **50% redemption cap.** It is enforced in the UI and API; the procedure shown only checks balance. Say so honestly if asked, and name the procedure as the place to add the third check.
3. **Bill number format.** The procedure builds `BILL-` plus a timestamp to the second. Two bills created in the same second would collide on the UNIQUE constraint (and be rejected), so a sequence or a per-day counter would be more robust.
4. **Loyalty rounding.** Points redeemed are subtracted 1:1 as rupees in `v_bill_totals`; `Loyalty_Redeem_Rate` in `Settings` is not consulted there.
5. **Transport security.** Local demo traffic is plain HTTP; production needs HTTPS and a secret key outside the code.

6. **Demo credentials** above are for the presentation only and must never be used in a real deployment.